

TrafficVision AI: Comprehensive Strategy & Development Guide

Project: Automated Traffic Violation Detection Platform

Core Technologies: Python, YOLOv8 (Vision), PaddleOCR (Text Reading), OpenCV (Image Processing), FastAPI (Backend), React.js (Dashboard)

1. The Main Idea (Why We Are Building This)

The Problem: Cities have thousands of CCTV cameras capturing millions of images daily. Human traffic police cannot manually watch every feed to catch violators. It is slow, prone to human fatigue, and highly inconsistent.

The Solution: TrafficVision AI acts as a "virtual traffic cop" that never sleeps. It is an automated software pipeline that looks at a photo or video frame, instantly identifies all vehicles, checks if anyone is breaking the rules (like riding without a helmet or jumping a red light), reads their license plate, and generates a ready-to-use digital penalty slip (challan) with annotated photo evidence.

The Goal for the Hackathon: Build a working prototype that successfully processes an uploaded traffic image, identifies specific violations, reads the number plate, and displays the result on a clean web dashboard in under a second.

2. How It Works: The Step-by-Step Pipeline

We process images in 5 clear steps. Think of it like an assembly line for digital evidence.

Step 1: Clean the Image (Preprocessing)

- **The Concept:** Cameras often capture bad photos due to glaring headlights, nighttime darkness, or rain. We need to "clean" the photo before the AI looks at it.
- **The Tech:** We use **OpenCV** to apply techniques like CLAHE (adjusting contrast) and denoising.

Step 2: Spot the Objects (Detection)

- **The Concept:** The system scans the cleaned image to find every vehicle, person, and rider. It draws an invisible "bounding box" around them.
- **The Tech:** We use **YOLOv8** (You Only Look Once), a state-of-the-art AI model known for being incredibly fast. It will classify objects into categories: Car, Bus, Motorcycle, Person, etc.

Step 3: Catch the Violation (Rule Engine)

- **The Concept:** Now that we know *where* everything is, we check for broken rules.
- **The Tech (How we detect specific things):**
 - *No Helmet:* We crop the box containing a motorcycle rider's head and pass it to a smaller AI to ask: "Is there a helmet here? Yes or No?"
 - *Triple Riding:* We count the number of "Person" boxes overlapping a single "Motorcycle" box. If Count ≥ 3 , flag a violation.
 - *Red Light/Stop Line:* We draw an invisible line on the camera view. If a car's box crosses this line while the light is red, flag it.

Step 4: Read the Number Plate (ANPR)

- **The Concept:** Once a violator is caught, we need to know who they are. The AI zooms in on the license plate and reads the text.
- **The Tech:** **PaddleOCR** extracts the text. We then use a "Regex Filter" (a text pattern checker) to ensure it looks like a valid Indian number plate (e.g., DL 4C AB 1234) to drop junk detections.

Step 5: Generate Evidence (Output)

- **The Concept:** The system paints colored boxes on the original image (e.g., a red box around the unhelmeted rider, a blue box around the license plate) and saves all the text data to a database.
- **The Tech:** **FastAPI** handles this logic, stores the text data in **PostgreSQL**, and sends it to our **React.js** web dashboard for the human operator to view.

3. Development Strategy (Hackathon Roadmap)

To avoid chaos, we will build this prototype in 5 agile phases. Do not build the web dashboard until the AI actually works.

- **Phase 1: The Base AI (Focus on Detection)**
 - *Goal:* Get YOLOv8 to successfully draw boxes around cars and bikes on a sample Indian traffic photo.
 - *Task:* Setup Python, load YOLOv8, and run a basic inference script.
- **Phase 2: The Violation Logic (Focus on Rules)**
 - *Goal:* Make the system smart enough to detect specific rules.
 - *Task:* Write the Python scripts for calculating overlap (for triple riding) and train/load a small classifier to detect helmets on cropped images.
- **Phase 3: Reading Plates (Focus on OCR)**
 - *Goal:* Successfully extract text from license plates.
 - *Task:* Crop the license plate using YOLO, enhance its resolution using OpenCV, and pass it to PaddleOCR. Print the text to the terminal.
- **Phase 4: Tying it Together (Backend Assembly)**
 - *Goal:* Turn scripts into a usable software service.
 - *Task:* Wrap the entire Phase 1, 2, and 3 pipeline into a **FastAPI** application. Create an endpoint where you can upload an image, and it returns a JSON file with all the

detected violations and base64 evidence images.

- **Phase 5: The Face of the App (Frontend Dashboard)**
 - *Goal:* Make it look good for the judges.
 - *Task:* Build a simple **React.js** web page with an "Upload Image" button, a table showing recent violations, and a modal to click and view the annotated evidence photos.

4. Coding Structure (Where Files Go)

Organize your GitHub repository exactly like this to keep the team from overwriting each other's work:

- `/api` -> Put your FastAPI server files here (main.py, route handlers).
- `/core/vision` -> Put your YOLOv8 scripts here (the code that detects vehicles).
- `/core/rules` -> Put your violation logic here (e.g., helmet_check.py, triple_riding.py).
- `/core/ocr` -> Put the PaddleOCR scripts here.
- `/database` -> Put your PostgreSQL connection files here.
- `/frontend` -> Put your React.js application here.
- `/test_images` -> Keep a folder of raw images to test the system during development.

5. Future Enhancements (Post-Hackathon Vision)

Judges want to know what this looks like as a real, deployed product. Here is the roadmap for the future:

Short-Term (Months 1-3)

- **Live Video Streams (RTSP):** Move from uploading static images to connecting directly to live CCTV camera IP feeds.
- **Vahan Database Integration:** Connect our system to the government vehicle registry API to automatically fetch the vehicle owner's name and address based on the license plate.

Medium-Term (Months 3-9)

- **Edge Computing:** Video files are huge. Sending live video over the internet to a cloud server is expensive. We will compress our AI models so they run directly on tiny computers (like Raspberry Pi or NVIDIA Jetson) attached to the camera pole itself.
- **Vehicle Re-identification:** Track a specific car across multiple different cameras across the city using its visual fingerprint, even if the license plate is dirty.

Long-Term (Months 9+)

- **Predictive Analytics:** Use historical violation data to predict *where* and *when* accidents are most likely to happen, allowing the city to deploy physical police units or ambulances preemptively.